ĐẶC TẢ THIẾT KẾ gemm_register_file

Mục lục

1 Tổng quan **2**
1.1 Tính năng chính . . . 2
1.2 Tham số . . . 2
1.3 Tín hiệu giao tiếp . . . 2
1.4 Tín hiệu AXI . . . 4

2 Đặc tả thanh ghi **5**

3 Mô tả chức năng **6**
3.1 Tổ chức chức năng . . . 6
3.2 Thanh ghi . . . 6
3.3 Timing . . . 8
3.4 Reset, hiện thực, kiểm chứng . . . 8

1. Tổng quan

1.1. Tính năng chính

Register File là đầu mối duy nhất giữa CPU và accelerator. Nó trả lời mọi truy cập MMIO trên AXI4-Lite, lưu các thanh ghi cấu hình, quyết định chấp nhận START hay START_CHAIN, chụp cấu hình đúng lúc chấp nhận, giữ cờ trạng thái DONE/ERROR, và đếm chu kỳ hiệu năng. Nó không biết gì về các pha: Main FSM báo lại khi tác vụ xong.

Chức năng

- Register map và bố trí bit là giao diện phần mềm được định nghĩa tại mục Đặc tả thanh ghi của spec tổng. Module này hiện thực giải mã, ghi byte, đọc lại và cập nhật trạng thái của giao diện đó
- Chỉ nhận START/START_CHAIN khi rảnh. Chụp cấu hình và báo start trước khi hoàn tất phản hồi ghi AXI. DONE/ERROR giữ đến tác vụ mới hoặc reset; MODE giữ chế độ gần nhất và ERR_CODE bằng 0 khi không lỗi.
- Chụp đủ 32 bit của `MATRIX_DIM`, ba thanh ghi địa chỉ cơ sở, `POST_CTRL`, `BIAS_ADDR`, `QUANT_M0`, `QUANT_SHIFT` và `DESC_PTR` khi nhận tác vụ; giữ nguyên tới lần chấp nhận kế tiếp.
- Các bộ đếm PERF xóa khi nhận tác vụ, đếm khi BUSY, bão hòa tại 0xFFFF_FFFF và giữ sau kết thúc. Khi các pha chồng nhau, tổng các bộ đếm pha có thể lớn hơn tổng thời gian tác vụ.
- Luôn trả OKAY; không bao giờ treo bus

1.2. Tham số

Bảng 1: Tham số

Tham số	Mặc định	Ý nghĩa
`P`	8	Ghi vào `HW_CONFIG[7:0]`
`HAS_CHAIN`	1	`HW_CONFIG[16]`; bật các offset 0x40 … 0x48
`HAS_PINGPONG`	1	`HW_CONFIG[17]`

1.3. Tín hiệu giao tiếp

Bảng 2: Tín hiệu giao tiếp của `gemm_register_file`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset tích cực mức thấp; được kích hoạt bất đồng bộ. Cạnh nhả reset được đồng bộ với clock tại top trước khi đưa tới các module.
`i_soft_clr`	1	Input	Xung xóa đồng bộ dài một chu kỳ do Main FSM phát. Xóa trạng thái điều khiển tại cạnh lên; phạm vi xóa dữ liệu được quy định trong mục Reset.
Register File → FSM			
`o_start_pulse`	1	Output	Xung một chu kỳ báo Register File vừa chấp nhận một tác vụ. Loại lệnh và bản chụp cấu hình hợp lệ cùng chu kỳ.

Bảng 2: Tín hiệu giao tiếp của `gemm_register_file` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`o_start_kind`	2	Output	Loại lệnh được chấp nhận. Bit 0 ứng với START; bit 1 ứng với START_CHAIN. Giá trị 01: tác vụ đơn; 10: chuỗi; 11: cấu hình lệnh sai.
`o_soft_req`	1	Output	Xung một chu kỳ yêu cầu Main FSM thực hiện soft reset. Phát khi lệnh ghi CTRL có SOFT_RESET = 1 và byte 0 được cho phép ghi.
`o_cfg_dim`	32	Output	Bản chụp đủ 32 bit của MATRIX_DIM. Bit [7:0]: M; [15:8]: N; [23:16]: K; [31:24]: dự trữ. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
`o_cfg_a_addr`	32	Output	Bản chụp đủ 32 bit của BASE_ADDR_IN. Địa chỉ byte của ma trận A. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
`o_cfg_b_addr`	32	Output	Bản chụp đủ 32 bit của BASE_ADDR_WEIGHT. Địa chỉ byte của ma trận B. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
`o_cfg_c_addr`	32	Output	Bản chụp đủ 32 bit của BASE_ADDR_OUT. Địa chỉ byte vùng kết quả C hoặc Y. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
`o_cfg_post_ctrl`	32	Output	Bản chụp đủ 32 bit của POST_CTRL. Bit 0 bật hậu xử lý; bit 1 bật ReLU. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
`o_cfg_bias_addr`	32	Output	Bản chụp đủ 32 bit của BIAS_ADDR. Địa chỉ byte của vector bias. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
`o_cfg_qm0`	32	Output	Bản chụp đủ 32 bit của QUANT_M0. Hệ số nhân lượng tử M0, INT32 có dấu. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
`o_cfg_qshift`	32	Output	Bản chụp đủ 32 bit của QUANT_SHIFT. Bit [4:0] là lượng dịch sh; giữ cả các bit cao để kiểm tra cấu hình. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
`o_cfg_desc_ptr`	32	Output	Bản chụp đủ 32 bit của DESC_PTR. Địa chỉ descriptor đầu chuỗi; bằng 0 khi không hỗ trợ chuỗi. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
FSM → Register File			
`i_soft_busy`	1	Input	Báo đang xử lý soft reset. Trong khoảng này Register File báo BUSY = 1 và không nhận tác vụ mới.
`i_done_pulse`	1	Input	Xung một chu kỳ báo toàn bộ tác vụ kết thúc thành công. Register File đặt DONE và xóa BUSY tại cạnh lên; không xuất hiện cùng err pulse.

Bảng 2: Tín hiệu giao tiếp của `gemm_register_file` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_err_pulse`	1	Input	Xung một chu kỳ báo toàn bộ tác vụ kết thúc với lỗi. Mã lỗi hợp lệ cùng chu kỳ; Register File đặt ERROR và xóa BUSY tại cạnh lên.
`i_err_code`	4	Input	Mã lỗi của tác vụ, được lấy mẫu cùng err pulse. Giá trị 1: cấu hình. Giá trị 2: đọc dữ liệu. Giá trị 3: ghi kết quả; 4: đọc descriptor; 5: tính năng không có.
`i_perf_load`	1	Input	Báo pha LOAD dữ liệu, không bao gồm FETCH đang hoạt động. Khi tín hiệu bằng 1 và tác vụ đang bận, bộ đếm hiệu năng tương ứng tăng tại cạnh lên.
`i_perf_comp`	1	Input	Báo pha COMPUTE đang hoạt động. Khi tín hiệu bằng 1 và tác vụ đang bận, bộ đếm hiệu năng tương ứng tăng tại cạnh lên.
`i_perf_store`	1	Input	Báo pha STORE_C đang hoạt động. Khi tín hiệu bằng 1 và tác vụ đang bận, bộ đếm hiệu năng tương ứng tăng tại cạnh lên.
`i_err_desc_addr`	32	Input	Địa chỉ descriptor lỗi được cung cấp cho thanh ghi ERR_DESC_ADDR tại offset 0x44. Bằng 0 ở chế độ đơn hoặc khi không hỗ trợ chuỗi.
`i_desc_done_cnt`	32	Input	Số descriptor đã hoàn tất STORE_C thành công, cung cấp cho thanh ghi DESC_DONE_CNT tại offset 0x48. Bằng 0 ở chế độ đơn hoặc khi không hỗ trợ chuỗi.

Giao tiếp

- Mọi port ra (`s_axi_*` ra và `o_*`) lấy từ thanh ghi
- `o_cfg_*` chỉ đổi ở cạnh clock chấp nhận tác vụ; ổn định suốt tác vụ
- `i_done_pulse` và `i_err_pulse` chỉ đến khi BUSY = 1 và không bao giờ cùng chu kỳ

1.4. Tín hiệu AXI

Bảng 3: Tín hiệu AXI

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`s_axi_awaddr`	32	Input	Địa chỉ byte của thanh ghi cần ghi. Chỉ giải mã bit [11:2]; bỏ qua bit [1:0]. Hợp lệ cùng AWVALID; giữ ổn định khi chờ bắt tay.
`s_axi_awvalid`	1	Input	Báo địa chỉ và thuộc tính yêu cầu ghi hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi AWREADY = 1.
`s_axi_awready`	1	Output	Báo bên nhận sẵn sàng nhận địa chỉ ghi. Bắt tay xảy ra tại cạnh lên khi AWVALID và AWREADY cùng bằng 1.
`s_axi_wdata`	32	Input	Word 32 bit cần ghi. Hợp lệ khi WVALID = 1; giữ nguyên trong lúc chờ WREADY.
`s_axi_wstrb`	4	Input	Cho phép ghi từng byte của thanh ghi RW. Bit b = 1 cho phép ghi byte b; bit b = 0 giữ byte đó. Các bit lệnh CTRL chỉ có hiệu lực khi bit 0 bằng 1.

Bảng 3: Tín hiệu AXI (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
s_axi_wvalid	1	Input	Báo dữ liệu ghi hợp lệ. Giữ valid và payload ổn định đến khi WREADY = 1.
s_axi_wready	1	Output	Báo bên nhận sẵn sàng nhận dữ liệu ghi. Giao dịch được nhận tại cạnh lên khi WVALID và WREADY cùng bằng 1.
s_axi_bresp	2	Output	Mã phản hồi ghi. Luôn bằng 2'b00 (OKAY), kể cả truy cập offset dự trữ. Hợp lệ cùng BVALID.
s_axi_bvalid	1	Output	Báo phản hồi ghi hợp lệ. Giữ valid và payload ổn định đến khi BREADY = 1.
s_axi_bready	1	Input	Báo bên nhận sẵn sàng nhận phản hồi ghi. Giao dịch được nhận tại cạnh lên khi BVALID và BREADY cùng bằng 1.
s_axi_araddr	32	Input	Địa chỉ byte của thanh ghi cần đọc. Chỉ giải mã bit [11:2]; bỏ qua bit [1:0]. Hợp lệ cùng ARVALID; giữ ổn định khi chờ bắt tay.
s_axi_arvalid	1	Input	Báo địa chỉ và thuộc tính yêu cầu đọc hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi ARREADY = 1.
s_axi_arready	1	Output	Báo bên nhận sẵn sàng nhận địa chỉ đọc. Bắt tay xảy ra tại cạnh lên khi ARVALID và ARREADY cùng bằng 1.
s_axi_rdata	32	Output	Word 32 bit đọc trả về. Hợp lệ khi RVALID = 1; giữ nguyên trong lúc chờ RREADY.
s_axi_rresp	2	Output	Mã phản hồi đọc. Luôn bằng 2'b00 (OKAY), kể cả truy cập offset dự trữ. Hợp lệ cùng RVALID.
s_axi_rvalid	1	Output	Báo dữ liệu đọc hợp lệ. Giữ valid và payload ổn định đến khi RREADY = 1.
s_axi_rready	1	Input	Báo bên nhận sẵn sàng nhận dữ liệu đọc. Giao dịch được nhận tại cạnh lên khi RVALID và RREADY cùng bằng 1.

2. Đặc tả thanh ghi

Offset, từng trường bit, quyền truy cập và reset được định nghĩa đầy đủ trong mục Đặc tả thanh ghi của spec tổng. Các thanh ghi RW lưu đủ 32 bit, kể cả bit dự trữ, và ghi theo byte strobe. Các bảng hành vi bên dưới quy định cách module thực hiện giao diện đó.

3. Mô tả chức năng

3.1. Tổ chức chức năng

3.1.1. `gemm_axil_slave`

Giao thức AXI4-Lite

- Thanh ghi giữ `aw_v`/`aw_addr` và `w_v`/`w_data`/`w_strb`. `s_axi_awready` = ¬`aw_v` ∧ ¬`s_axi_bvalid`; `s_axi_wready` = ¬`w_v` ∧ ¬`s_axi_bvalid`. AW và W được nhận theo thứ tự bất kỳ
- `wr_fire` = `aw_v` ∧ `w_v` ∧ ¬`s_axi_bvalid`: lệnh ghi có hiệu lực ở cạnh clock cuối chu kỳ đó; cùng cạnh: `aw_v`, `w_v` ← 0, `s_axi_bvalid` ← 1, `s_axi_bresp` = 0
- `s_axi_bvalid` giữ tới `s_axi_bready` . Tối đa một lệnh ghi đang xử lý
- `s_axi_arready` = ¬`s_axi_rvalid`. Ở chu kỳ bắt tay AR (`rd_fire`): `s_axi_rdata` ← giá trị đọc tính từ trạng thái đầu chu kỳ, `s_axi_rvalid` ← 1; giữ tới `s_axi_rready`
- Chỉ dùng bit địa chỉ [11:2] . Đọc và ghi cùng chu kỳ được phép; đọc trả giá trị trước lệnh ghi

3.2. Thanh ghi

Bảng 4: Hành vi từng offset (`gemm_rf_regs`)

Offset	Tên	Đọc ra	Ghi	Soft reset
0x00	CTRL	0	Lệnh, chỉ khi `strb[0]` = 1	—
0x04	STATUS	(STATUS = {24'b0, `ERR_CODE`, MODE, ERROR, `busy_flag`, DONE}. `i_done_pulse`: DONE ← 1, `busy_flag` ← 0. `i_err_pulse`: ERROR ← 1, `ERR_CODE` ← `i_err_code`, `busy_flag` ← 0. Như vậy BUSY về 0 đúng ở cạnh clock DONE hoặc ERROR được đặt), (Khi `i_soft_busy` = 1, STATUS đọc ra 0x0000_0002 (chỉ BUSY), nên tại mọi thời điểm tối đa một trong BUSY, DONE, ERROR bằng 1)	Bỏ qua	Về 0
0x08 … 0x24	8 thanh ghi R/W	Giá trị lưu, đủ 32 bit	Theo byte strobe	Giữ
0x28	HW_CONFIG	{0x10, 6'b0, `HAS_PINGPONG`, `HAS_CHAIN`, 8'd32, 8'(P)}	Bỏ qua	—
0x2C … 0x38	`PERF_*`	Bộ đếm	Bỏ qua	Về 0
0x40	`DESC_PTR`	Giá trị lưu (`HAS_CHAIN`), ngược lại 0	Theo byte strobe (`HAS_CHAIN`), ngược lại bỏ qua	Giữ
0x44, 0x48	`ERR_DESC_ADDR`, `DESC_DONE_CNT`	`i_err_desc_addr`, `i_desc_done_cnt` (`HAS_CHAIN`), ngược lại 0	Bỏ qua	FSM xóa

Bảng 4: Hành vi từng offset (`gemm_rf_regs`) (tiếp theo)

Offset	Tên	Đọc ra	Ghi	Soft reset
Khác	dự trữ	0	Bỏ qua	—

Bảng 5: Lệnh ghi CTRL có `strb[0]` = 1

Bit ghi	Hành vi ở cạnh clock của `wr_fire`
SOFT_RESET = 1	`o_soft_req` ← 1 (xung ở chu kỳ sau). START, START_CHAIN bị bỏ qua
START ∨ START_CHAIN, và rảnh	Chấp nhận
START ∨ START_CHAIN, đang bận	Bỏ qua, không đổi gì
Không bit nào	Không có tác dụng

Chấp nhận và trạng thái

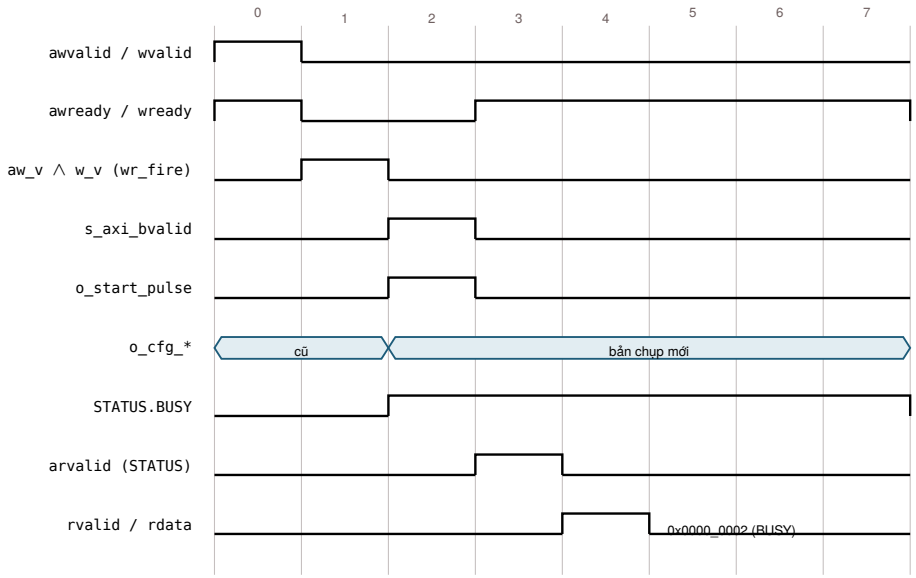
- "Rảnh" = ¬`busy_flag` ∧ ¬`i_soft_busy` ∧ ¬`o_soft_req`. Khi chấp nhận, ở cùng cạnh clock: `o_cfg_*` ← các thanh ghi R/W và `DESC_PTR`; `busy_flag` ← 1; DONE, ERROR, `ERR_CODE` ← 0; MODE ← ¬START ∧ START_CHAIN; `PERF_*` ← 0; `o_start_pulse` ← 1, `o_start_kind` ← {START_CHAIN, START}
- STATUS = {24'b0, `ERR_CODE`, MODE, ERROR, `busy_flag`, DONE}. `i_done_pulse`: DONE ← 1, `busy_flag` ← 0. `i_err_pulse`: ERROR ← 1, `ERR_CODE` ← `i_err_code`, `busy_flag` ← 0. Như vậy BUSY về 0 đúng ở cạnh clock DONE hoặc ERROR được đặt
- Khi `i_soft_busy` = 1, STATUS đọc ra 0x0000_0002 (chỉ BUSY), nên tại mọi thời điểm tối đa một trong BUSY, DONE, ERROR bằng 1
- `i_soft_clr`: `busy_flag`, DONE, ERROR, MODE, `ERR_CODE`, `PERF_*` ← 0 . Thanh ghi R/W, `DESC_PTR`, `o_cfg_*` giữ nguyên. Ưu tiên: `i_soft_clr` > chấp nhận > `i_done_pulse`/`i_err_pulse`
- `o_start_pulse`, bản chụp và `busy_flag` = 1 có hiệu lực ở cùng chu kỳ `s_axi_bvalid` lên 1, nên mọi lần đọc STATUS bắt đầu sau khi CPU nhận B đều thấy tác vụ mới

3.2.1. `gemm_rf_perf`

Bộ đếm hiệu năng

- Ở mỗi chu kỳ có `busy_flag` = 1 (không phải chu kỳ chấp nhận): `PERF_TOTAL++`; `PERF_LOAD++` nếu `i_perf_load`; `PERF_COMPUTE++` nếu `i_perf_comp`; `PERF_STORE++` nếu `i_perf_store`. Mỗi bộ đếm bão hòa ở 0xFFFF_FFFF
- Bộ đếm về 0 khi chấp nhận tác vụ, khi `i_soft_clr` và khi `rst_n`; giữ giá trị sau khi tác vụ kết thúc

3.3. Timing



Hình 1: Ghi START (AW, W cùng chu kỳ 0, `bready` = 1) rồi đọc STATUS. Lệnh ghi có hiệu lực ở cạnh clock cuối chu kỳ 1; B, xung start và BUSY cùng xuất hiện ở chu kỳ 2

Bảng 6: Độ trễ

Sự kiện	Chu kỳ
Bắt tay AW và W đều xong ở t → `wr_fire`	t + 1
`wr_fire` ở t → `s_axi_bvalid`, `o_start_pulse`, bản chụp, BUSY	t + 1
Bắt tay AR ở t → `s_axi_rvalid`	t + 1
`i_done_pulse` / `i_err_pulse` ở t → STATUS mới	t + 1
Thông lượng ghi liên tục	1 lệnh mỗi 3 chu kỳ khi `bready` = 1

3.4. Reset, hiện thực, kiểm chứng

Reset

- `rst_n` = 0: mọi thanh ghi về giá trị reset trong mục Đặc tả thanh ghi của spec tổng; `aw_v`, `w_v`, `s_axi_bvalid`, `s_axi_rvalid`, `o_start_pulse`, `o_soft_req` về 0; `o_cfg_*` về 0
- `i_soft_clr`: `busy_flag`, DONE, ERROR, MODE, `ERR_CODE`, `PERF_*` ← 0 . Thanh ghi R/W, `DESC_PTR`, `o_cfg_*` giữ nguyên. Ưu tiên: `i_soft_clr` > chấp nhận > `i_done_pulse`/`i_err_pulse` . Giao dịch AXI4-Lite đang dở không bị ảnh hưởng

Hiện thực

- SystemVerilog tổng hợp được, không latch. Mux đọc là `case` trên `araddr[11:2]`
- Với `HAS_CHAIN` = 0, thanh ghi `DESC_PTR` được loại bằng `generate`

Assertion bắt buộc (SVA)

- `s_axi_bvalid` ∧ ¬`s_axi_bready` → `s_axi_bvalid` ở chu kỳ sau; tương tự `rvalid` và `rdata` không đổi
- `o_start_pulse` → ¬`busy_flag` ở chu kỳ trước
- Tối đa một trong STATUS[0], STATUS[1], STATUS[2] bằng 1
- `busy_flag` → `$stable(o_cfg_*)`

- `i_done_pulse` $\vee$ `i_err_pulse` $\rightarrow$ `busy_flag`

> **KIỂM CHỨNG BẰNG MÔ HÌNH**
> `gemm_regfile_model.py`: 200 lượt $\times$ 60 lệnh ngẫu nhiên (AW và W lệch nhau 0 … 3 chu kỳ, `bready` ngẫu nhiên, byte strobe ngẫu nhiên, offset dự trữ, START/SOFT_RESET xen kẽ với FSM giả lập kết thúc tác vụ sau thời gian ngẫu nhiên): đọc lại thanh ghi R/W đúng từng byte, offset dự trữ đọc 0, STATUS không bao giờ có hai cờ cùng lúc; 300 lượt riêng kiểm (Register File phát `start_pulse` và chụp cấu hình trước khi trả phản hồi B cho lệnh ghi CTRL; mọi lần đọc STATUS sau khi lệnh ghi hoàn tất đều thấy tác vụ mới).